

inform the client whenever an event of interest occurs. Such a model reduces the bandwidth requirement or the polling cycle requirement, as the client no longer requests for new data regularly.

Notifier chains

Linux uses notifier chains to inform asynchronous events or status, through the function calls registered. The data structure is defined in `include/linux/notifier.h`:

```
struct notifier_block {
    int (*notifier_call)(struct notifier_block *,
unsigned long, void *);
    struct notifier_block *next;
    int priority;
};
```

The notifier data structure is a simple linked list of function pointers. The function pointers are registered with 'functions' that are to be called when an event occurs. Each module needs to maintain a notifier list. The functions are registered to this notification list.

The notification module (publisher) maintains a list head that is used to manage and traverse the notifier block list. The function that subscribes to a module is added to the head of the module's list by using the `xxxxxx_notifier_chain_register` API and deletion from the list is done using `xxxxxx_notifier_chain_unregister`.

When an event occurs which is of interest to a particular list, then the `xxxxxx_notifier_call_chain` API is used to traverse the list and service the subscribers. The 'xxxxxx' in the above APIs represents the type of notifier chains.

Let us now look at the different types of notifier chains in the following section.

Types of notifier chains

Notifier chains are broadly classified based on the context in which they are executed and the lock/protect mechanism of the calling chain. Based on the need of the module, the notifiers can be executed in the process context or interrupt/atomic context. Thus, notifier chains are classified into four types:

- **Atomic notifier chains:** As the name indicates, this notifier chain is executed in interrupt or atomic context. Normally, events that are time critical, use this notifier. This also means it is a non-blockable call. Linux modules use atomic notifier chains to inform watchdog timers or message handlers. For example, `register_keyboard_notifier` uses `atomic_notifier_chain_register` to get called back on keyboard events. This notifier is usually called from the interrupt context.
- **Blocking notifier chains:** A blocking notifier chain runs in the process context. The calls in the notification list could be blocked as it runs in the

process context. Notifications that are not highly time critical could use blocking notifier chains. Linux modules use blocking notifier chains to inform the modules on a change in QOS value or the addition of a new device. For example, `usb_register_notify` uses `blocking_notifier_chain_register` to inform either USB devices or buses being added or removed.

- **Raw notifier chains:** A raw notifier chain does not manage the locking and protection of the callers. Also, there are no restrictions on callbacks, registration, or de-registration. It provides flexibility to the user to have individual lock and protection mechanisms. Linux uses the raw notifier chain in low-level events. For example, `register_cpu_notifier` uses `raw_notifier_chain_register` to pass on CPU going up/down information.
- **SRCU notifier chains:** Sleepable Read Copy Update (SRCU) notifier chains are similar to the blocking notifier chain and run in the process context. It differs in the way it handles locking and protection. The SRCU methodology brings in less overhead when we notify the registered callers. On the flip side, it consumes more resource while unregistering. So it is advisable to choose this methodology where we use the notifier call often and where there's very little requirement for removing from the chain. For example, Linux module uses `srcu_notifier_chain_register` in CPU frequency handling.

Using notifier chains

Let us consider two modules: a publisher and a subscriber. The publisher module has to maintain and export a 'notification head'. Generally, this is exported through an interface function that helps the subscriber to register itself with the publisher. The subscriber has to provide a callback function through `notifier_block`. Let us now look at how a publisher and a subscriber work using blocking notifier chains.

Assume a scenario in which an action needs to be taken by a module when a USB device is plugged into the kernel. Any USB activity is first detected by the USB core of the Linux kernel. The USB core has to 'publish' a notification list head to inform new USB devices of activity in the kernel. Thus the USB core becomes the publisher.

The USB core publishes its notification list through the following interface function and the notifier list data structure (snippet of `drivers/usb/core/notify.c` file):

```
18 static BLOCKING_NOTIFIER_HEAD(usb_notifier_list);
19
20 /**
21  * usb_register_notify - register a notifier callback whenever a
    usb change happens
22  * @nb: pointer to the notifier block for the callback events.
23  */
```